Kangaroo: Caching Billions Of Tiny Objects On Flash

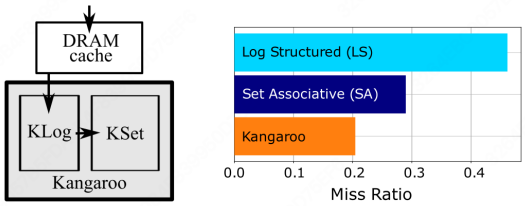
March 26, 2026

1 Introduction

Flash's main challenge is its limited write endurance; i.e., flash can only be written so many times before wearing out. Wearout is especially problematic for tiny objects because flash can be read and written only at multi-KB granularity. For example, writing a 100 B object may require writing a 4 KB flash page, amplifying bytes written by 40× and rapidly wearing out the flash device. Thus, cost also argues for minimizing excess bytes written to flash.

The Problem. Prior flash-cache designs either use too much DRAM or write flash too much.

- LSM: tracking billions of tiny objects requires a large index, and even a heavily optimized index needs large amounts of DRAM
- Set-associative caches operate by hashing objects' keys into distinct "sets," much like CPU caches. Admitting a single small object to the cache requires re-writing an entire set, significantly amplifying the number of bytes written to the flash device.



(a) Overview. ***(b)*** Kangaroo reduces misses by 29%.

Fig. 1. (a) High-level illustration of Kangaroo's design. (b) Miss ratio achieved on a production trace from Facebook by different flash-cache designs on a 1.9 TB drive with a budget of 16 GB DRAM and three device-writes per day. Prior designs are constrained by either DRAM or flash writes, whereas Kangaroo's design balances these constraints to reduce misses by 29%.

Figure 1

- To avoid a large DRAM index, Kangaroo organizes the bulk of cache capacity as a set-associative cache, called *KSet*.
- To reduce flash writes, Kangaroo places a small (e.g., 5% of flash) log-structured cache, called *KLog*, in front of KSet. KLog buffers many objects, looking for objects that map to the same set in KSet (i.e., hash collisions), so that each flash write to KSet can insert multiple objects.

Our insight is that even a small log will yield many hash collisions, so only a small amount of extra DRAM (for KLog's index) is needed to significantly reduce flash writes (in KSet).

2 Background and related work

2.1 Tiny objects are important and numerous

Tiny objects are prevalent in many large-scale systems:

- At Facebook, small objects are prevalent in the social graph. For example, the average social-graph edge size is under 100 B. Across edges, nodes, and other objects, the average object size is less than 700 B. This has led to the development of a dedicated flash caching system for small objects.
- At Twitter, tweets are limited to 280 B, and the average tweet is less than 33 characters. Due to the massive and growing number of tweets, Twitter seeks a cost-effective caching solution.

- At Microsoft Azure, a growing use case is processing updates from sensor data, such as from IoT devices in Azure Streaming Analytics. Before an update can be processed (e.g., to trigger a real-time action), the server must fetch metadata (the sensor's unit of measurement, geolocation, owner, etc.) with an average size of 300 B. For efficiency and availability, it caches the most popular metadata. Another use case arises in search advertising, where Azure caches predictions and other results

2.2 Caching tiny objects in flash is hard

DRAM is expensive

Challenges:

- flash has *limited write endurance*, which means there is a limit on the number of writes before the flash device wears out and must be replaced
- *write amplification*: Flash devices suffer from both device-level write amplification and application-level write amplification

Device-level write amplification (dlwa) occurs when the flash translation layer (FTL) writes more flash pages than asked for by the storage application (e.g., file system, database, or cache). Current flash drives implement the age-old block-storage interface, wherein hosts read and write logical blocks in a numerical logical-block address (LBA) namespace. Generally, the internal flash-page size and the external logical-block size are the same, with 4 KB being common, even though the flash device can only erase pages in much larger (e.g., 256 MB) "erase blocks". Most dlwa is caused by cleaning activity that copies live pages elsewhere before an erase block is cleared.

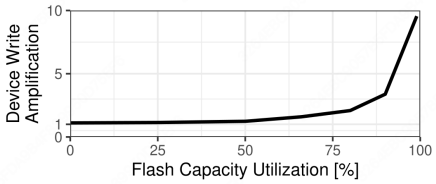


Fig. 2. The effect of flash over-provisioning on device-level write amplification (DLWA) of random writes of various sizes. DLWA increases as over-provisioning decreases.

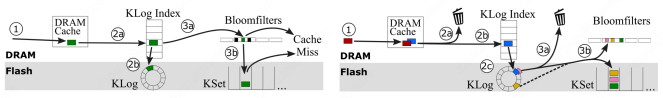
Figure 2

Generally speaking, dlwa worsens as more of the raw flash capacity is utilized and as access patterns consist more of small, random writes. A common approach to reduce dlwa is over-provisioning, i.e., only exposing a fraction of the raw flash capacity in the LBA namespace, so that cleaning tends to find fewer live pages in victim erase blocks. Fig. 2 shows dlwa vs. utilized capacity for random 4 KB writes to a 1.9 TB flash drive. As expected, dlwa significantly increases as over-provisioning decreases, from ≈1× at 50% utilization to ≈10× at 100% utilization.

Application-level write amplification (alwa) occurs when the storage application re-writes some of its own data as part of its storage management. One form of this is akin to FTL cleaning, such as cleaning in log-structured file systems or compaction in log-structured merge trees. Another form is caused by having to write an entire logical block. To write a smaller amount of data, the application must read the block, install the new data, and then write the entire block. For example, installing 1 KB of new data into a 4 KB logical block involves rewriting the other 3 KB, giving alwa of 4×. Ideally, the unmodified data in the block would not have been rewritten.

2.3 Current approaches and related work

3 Kangeroo Overview and Motivation



(a) A lookup in Kangaroo. ***(b)*** Insertion and eviction in Kangaroo.

Fig. 3. Overview of Kangaroo. Objects first go to a tiny DRAM cache; then KLog, a small on-flash log-structured cache with an in-DRAM index; and finally KSet, a large on-flash set-associative cache. KLog minimizes flash writes, and KSet minimizes DRAM usage.

Figure 3

Lookup:

1 lookups first check the DRAM cache, which is very small (<1% of capacity).

2 If the requested key is not found, requests next check KLog (≈5% of capacity). KLog maintains a DRAM index to track objects stored in a circular log on flash.

3 If the key is not found in KLog's index, requests check KSet (≈95% of capacity). KSet has no DRAM index; instead, Kangaroo hashes the requested key to find the set (i.e., the LBA(s) on flash) that might hold the object.

3a If the requested key is not in the small, per-set Bloom filter, the request is a miss.

3b Otherwise, the object is probably on flash, so the request reads the LBA(s) for the given set and scans for the requested key.

Insertion:

1 Newly inserted items are first written to the DRAM cache. This likely pushes some objects out of the DRAM cache, where they are either

2a dropped by KLog's pre-flash admission policy or

2b added to KLog's DRAM index and

2c appended to KLog's flash log (after buffering in DRAM to batch many insertions into a single flash write).

inserting objects to KLog will push other objects out of KLog, which are either

3a dropped by another admission policy or

3b inserted into KSet.

Insertions to KSet operate somewhat differently than in a conventional cache. For any object moved from KLog to KSet, Kangaroo moves *all objects in KLog that map to the same set* to KSet, no matter where they are in the log. Doing this amortizes flash writes in KSet, significantly reducing Kangaroo's alwa.

- KSet eliminates the DRAM index by hashing objects' keys to restrict their possible locations on flash. But KSet alone suffers too much write amplification, as every tiny object writes a full 4 KB page when admitted
- KLog comes to the rescue, serving as a write-efficient staging area in front of KSet, which Kangaroo uses to amortize KSet's writes.

On top of this basic design, Kangaroo introduces three techniques to minimize DRAM usage, minimize flash writes, and reduce cache misses.

1. Kangaroo's partitioned index for KLog can efficiently find all objects in KLog mapping to the same set in KSet, and is split into many independent partitions to minimize DRAM usage.

2. Kangaroo's threshold admission policy between KLog and KSet only admits objects to KSet when at least n objects in KLog map to the same set, reducing alwa by $\geq n\times$.

3. Kangaroo's "RRIParoo" eviction improves hit ratio in KSet by approximating RRIP, a state-of-the-art eviction policy, while only using a single bit of DRAM per object.

Theoretical foundations: We develop a Markov model of Kangaroo's basic design, including threshold admission. This model rigorously demonstrates that Kangaroo can greatly reduce alwa compared to a set-only design, without any increase in miss ratio.

Formally, we assume the commonly used independent reference model, in which objects are referenced independently with fixed probability per object. However, we make no assumptions about the object popularity distribution, so Theorem 3.1 holds across any popularity distribution (uniform, Zipfian, etc.). Suppose that KLog contains q objects; KSet contains s sets with w objects each; objects are admitted to flash with a p probability; and objects are only admitted to KSet if at least n new objects are being inserted.

Theorem 3.1. *Kangaroo's app-level write amplification is*

$$ALWA_{Kangaroo} = p\left(1 + \frac{w\overline{F}_X(n)}{\overline{F}_X(1)\mathbb{E}[X|X \geq n]}\right)$$

where $X \sim Binomial(q, 1/s)$ *and* $\overline{F}_X(n) = \sum_{i=1}^{n} \mathbb{P}[X = i]$ *is the probability of a set being re-written. Furthermore, the probability of admitting an object to KSet is* $\mathbb{P}[X \geq n \mid X \geq 1]$

4 Kangaroo Design and Implementation

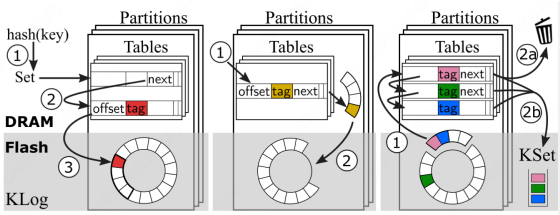
4.1 Pre-flash admission to KLog

It has a pre-flash admission policy that can be configured to randomly admit objects to KLog with probability p, decreasing Kangaroo's write rate proportionally without additional DRAM overhead.

4.2 KLog

Three operations: Lookup, Insert, and Enumerate-Set. Enumerate-Set allows KLog to find all objects mapping to the same set in KSet.

Operation. KLog writes objects to a circular buffer on flash in large batches and tracks objects via an index kept in DRAM. To support Enumerate-Set efficiently, KLog's index is implemented as a hash table using separate chaining. Each index entry contains an `offset` to locate the object in the flash log, a `tag` (partial hash of the object's key), a `next-pointer` to the next entry in the chain (for collision resolution), eviction-policy metadata, and a `valid` bit.



(a) KLog lookup. *(b)* KLog insertion. *(c)* KLog eviction.

Fig. 4. Overview of KLog operations.

Figure 4

Lookup:

1. KLog determines which bucket it belongs to by computing the object's set in KSet.
2. KLog traverses index entries in this bucket, ignoring invalid entries, until a tag matches a hash of the key. If there is no matching tag, KLog returns a miss.
3. KLog reads the flash page at offset in the log. After confirming a full key match, KLog returns the data and updates eviction-policy metadata.

Insert:

1. KLog creates an index entry, adds it to the bucket corresponding to the key's set in KSet, and appends the object to an in-DRAM buffer. The on-flash circular log is broken into many **segments**, one of which is buffered in DRAM at a time.
2. Once the segment buffer is full, it is written to flash.

KLog is structured internally as multiple partitions. Each partition is an independent log-structured cache with its own flash log and DRAM index. Moreover, each partition's index is split into multiple tables, each an independent hash table.

This partitioned structure reduces DRAM usage, as described next, but otherwise changes the operation of KLog little. The table and partition are inferred from an object's set in KSet. Hence, all objects in the same set will belong to the same partition, table, and bucket; and operations work as described above within each table.

	Component	Naïve Log-Only	Naïve Kangaroo	Kangaroo
KLog Index	`offset`	29 b	25 b	19 b
	`tag`	29 b	29 b	9 b
	`next-pointer`	64 b	64 b	16 b
	Eviction metadata	67 b	58 b	3 b
	`valid`	1 b	1 b	1 b
	Sub-total	*190 bits/obj*	*177 bits/obj*	*48 bits/obj*
KSet	Bloom filter	–	3 b	3 b
	Eviction	–	5 b	1 b
	Sub-total	–	*8 bits/obj*	*4 bits/obj*
Overall	Index buckets	≈ 3.1 b	≈ 3.1 b	≈ 0.8 b
	Log size	100% = 181 b	5% = 8.9 b	5% = 2.4 b
	Set size	0%	95% = 7.6 b	95% = 3.8 b
	Total	**193.1 bits/obj**	**19.6 bits/obj**	**7.0 bits/obj**

Table 1. Breakdown of DRAM per object for a 2 TB cache, comparing Kangaroo to a naïve log-structured cache and Kangaroo with a naïve log index. Bucket and LRU overhead assume 200 B objects.

Figure 5

The flash offset must be large enough to identify which page in the flash log contains the object, which requires $\log_2(LogSize/4\text{KB})$ bits. By splitting the log into 64 partitions, KLog reduces $LogSize$ by 64× and saves 6b in the pointer.

The tag size determines the false-positive rate in the index; i.e., a smaller tag leads to higher read amplification. KLog splits the index into 2^{20} tables. Since the table is inferred from the key, all keys in one table effectively share 20b of information, and KLog can use a much smaller tag to achieve the same false positive rate as the naïve design

KLog's structure also reduces the `next-pointer` size. We only need to know the offset into memory allocated to the object's index table. Thus, rather than using a generic memory pointer, we can store a 16b offset, which allows up to 2^{16} items per table. KLog can thus index 2^{36} items as parameterized (12.5 TB of flash with 200B objects), which can be increased by splitting the index into more tables.

In a naïve cache using LRU eviction, each entry keeps a pointer to adjacent entries in the LRU list. This requires $2 \cdot \log_2(LogSize/ObjectSize)$ bits. In contrast, Kangaroo's RRIParoo policy is based on RRIP and only needs 3b per object in KLog (and even less in KSet).

Finally, each bucket in KLog's index requires one pointer for the head of the chain. In naïve logs, this is a 64b pointer. In KLog, it is a 16b offset

into the table's memory. KLog allocates roughly one bucket per set in KSet. With 4 KB sets and 200 B objects, the per-object DRAM overhead is 3.1 b (Naïve) or 0.8 b (KLog) per object.

All told, KLog's partitioned structure reduces the per-object metadata from 190 b to 48 b per object, a 3.96× savings vs. the naïve design. Compared to prior index designs, KLog's index uses slightly more DRAM per object than the state- of-the-art (30 b per object in Flashield), but it supports Enumerate-Set and has fewer false positives. Most importantly, KLog only tracks ≈5% of objects in Kangaroo, so indexing overheads are just 3.2 b per object. Adding KSet's DRAM overhead gives a total of 7.0 b per object, a 4.3× improvement over the state-of-the-art.

4.3 KLog → KSet: Minimizing flash writes

Moving objects from KLog to KSet. A background thread keeps one segment free in each log partition. This thread flushes segments from the on-flash log in FIFO order, moving objects from KLog to KSet as shown in 4(c).

For each victim object in the flushed segment, this thread

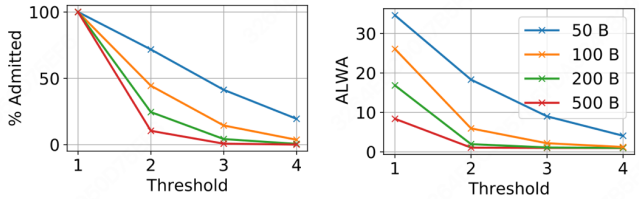
1 call Enumerate-Set to find all other objects in KLog

2a if there are not enough objects to move, the victim object is dropped, or if popular, is re-admitted to KLog

2b otherwise, the victim object and all other objects returned by Enumerate-Set are moved from KLog to KSet in a single flash write

Instead of flushing one segment at a time, one could fill the entire log and then flush it completely. But this leaves the log half-empty, on average. Flushing one segment at a time keeps KLog's capacity utilization high, empirically 80–95%.

Threshold admission to KSet. Kangaroo adds an admission policy between KLog and KSet that sets a threshold, n, of objects required to write a set in KSet. If Enumerate-Set(x) returns fewer than n objects, then x is not admitted to KSet.



(a) Percent of objects admitted. ***(b)*** ALWA.

Fig. 5. Modeled (a) admission percentage and (b) ALWA for Kangaroo with different threshold values and object sizes, assuming 4 KB sets and KLog w/ 5% of capacity.

Figure 6

Fig. 6 shows the effect of thresholding on alwa and KSet's admission probability for different object sizes using Theorem 3.1, keeping KLog at 5% of cache size.

To avoid unnecessary misses to popular objects that do not meet the threshold when moving from KLog to KSet, Kangaroo readmits any object that received a hit during its stay in KLog back to the head of the log.

4.4 KSet

KSet employs a set-associative cache design similar to CacheLib's Small Object Cache. This design splits the cache into many sets, each holding multiple objects; by default, each set is 4 KB, matching flash's read and write granularity. KSet maps an object to a set by hashing its key.

To reduce unnecessary flash reads, KSet keeps a small Bloom filter in DRAM built from all the keys in the set. These Bloom filters are sized to achieve a false positive rate of about 10%. Whenever a set is written, the Bloom filter is reconstructed to reflect the set's contents.

RRIParoo: Usage-based eviction without a DRAM index. Usage-based eviction policies can significantly improve miss ratio, effectively doubling the cache size (or more) without actually adding any resources. Unfortunately, implementing these policies on set-associative flash caches is hard, as such policies involve per-object metadata that must be updated whenever an object is accessed. Since KSet has no DRAM index to store metadata and cannot update on-flash metadata without worsening alwa, it is not obvious how to implement a usage-based eviction policy. For these reasons, most flash caches use FIFO eviction, which keeps no per-object state. Unfortunately, FIFO significantly increases miss ratio because popular objects continually cycle out of the cache.

Kangaroo introduces RRIParoo, a new technique to efficiently support usage-based eviction policies in flash caches. Specifically, RRIParoo implements RRIP, a state-of-the- art eviction policy originally proposed for processor caches, while using only ≈ 1 bit of DRAM per object and without any additional flash writes.

Background: How RRIP works. RRIP is essentially a multi-bit clock algorithm: RRIP associates a small number of bits with each object (3 bits in Kangaroo), which represent reuse predictions from near reuse (000) to far reuse (111). Objects are evicted only once they reach far. If there are no far objects when something must be evicted, all objects' predictions are incremented until at least one is at far. Objects are promoted to near (000) when they are accessed. Finally, RRIP inserts new objects at long (110) so they will be evicted quickly, but not immediately, if they are not accessed again. This insertion policy handles scans that can degrade LRU's performance.

RRIParoo's key ideas. There are two ideas to support RRIP in KSet.

1. rather than tracking all of RRIP's predictions in a DRAM index, RRIParoo stores the eviction metadata in flash and keeps only a small portion of it in DRAM

2. to reduce DRAM metadata to a single bit, we observe that RRIP only updates predictions upon eviction (incrementing predictions towards far) and when an object is accessed (promoting to near).

Our insight is that, so long as KSet tracks which objects are accessed, *promotions can be deferred to eviction time*, so that all updates to on-flash RRIParoo metadata are only made at eviction, when the set is being re-written anyway. Hence, since KSet can track whether an object has been accessed using only single bit in DRAM, KSet achieves the hit-ratio of a state-of-the-art eviction policy with one-third of the DRAM usage (1 b vs. 3 b).

1. 1 bit in dram

2. 3 bit in flash

3. Deferred Update Strategy

   (a) access: set bit to 1 in dram

   (b) eviction: if bit is 1, set prediction to 000; otherwise, increment prediction

RRIParoo operation. RRIParoo allocates enough metadata to keep one DRAM bit per object on average; e.g., 40b for 4 KB sets and 100B objects.

Objects use the bit corresponding to their position in the set (e.g., the i th object uses the i th bit), so there is no need for an index. If there are too many objects, RRIParoo does not track hits for the objects closest to near, as they are least likely to be evicted.

Kangaroo also uses RRIP to merge objects from KLog. Tracking hits in KLog is trivial because it already has a DRAM index. Objects are inserted into KLog at long prediction, and their predictions are decremented towards near on each subsequent access. Then, when moving objects from KLog to KSet, KSet sorts objects from near to far and fills up the set in this order until out of space, breaking ties in favor of objects already in KSet.

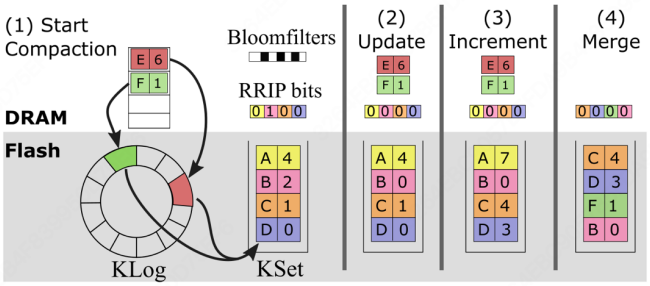


Fig. 6. RRIParoo implements RRIP eviction with only ≈1 b in DRAM per object and no additional flash writes.

Figure 7

Example: Figure 7.

1. We start when KLog flushes a segment containing object F, which maps to a set in KSet. KLog finds a second object, E, elsewhere in the log that also maps to this set. Meanwhile, the set contains objects A, B, C, and D with the RRIP predictions shown on flash, and B has received a hit since the set was last re-written, as indicated by bits in DRAM.

2. Since B received a hit, we promote its RRIP prediction to near and clear the bits in DRAM.

3. Since no object is at far, we increment all objects' predictions by 3, whereupon object A reaches far.

4. Finally, we fill up the set by merging objects in DRAM and flash in prediction-order until the set is full. The set now contains C, F, D, and C; A was evicted, and E stays in KLog for now since its KLog segment

was not flushed. (The set on flash is only written once, after the above procedure completes.)

5 Evaluation

Finding:

1. Kangaroo reduces misses by 29% under realistic system constraints.
2. Kangaroo improves the Pareto frontier when varying constraints.
3. In a production deployment, Kangaroo reduces flash-cache misses by 18% at equal write rate and reduces write rate by 38% at equal miss ratios.

5.1 Experimental setup

Comparison:

1. cachelib's small object cache (SA) that serves the Facebook Social Graph
2. optimistic version of a log-structured cache (LS) with a full DRAM index

Workload: sampled 7-day traces from Facebook and Twitter. Average object sizes of 291B aand 271B respectively.

5.2 Main result: Kangaroo significantly reduces misses vs. prior cache designs under realistic constraints

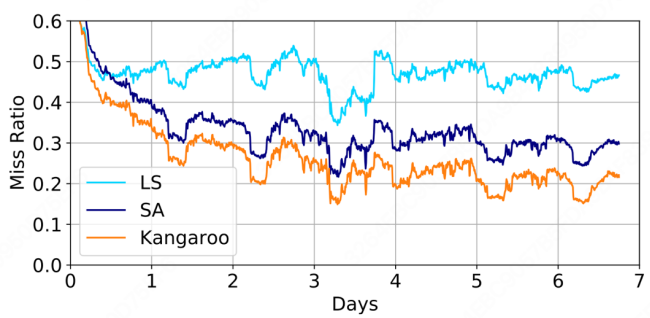
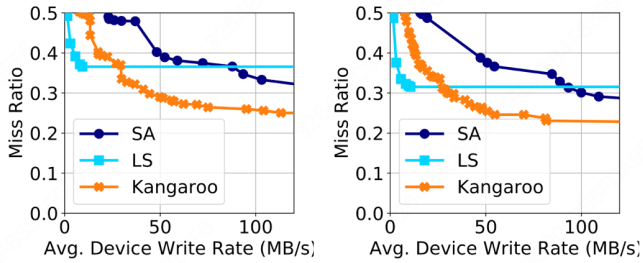


Fig. 7. Miss ratio for all three systems over a 7-day Facebook trace. All systems are run with 16 GB DRAM, a 1.9 TB drive, and with write rates less than 62.5 MB/s.

Figure 8

	Kangaroo	LS	SA
peak throughput(get/s)	158k	172k	168k
p99 latency (us)	736	229	699

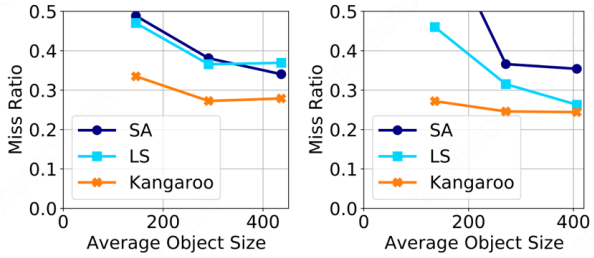
5.3 Kangaroo performs well as contraints change



(a) Facebook *(b)* Twitter

Fig. 8. Pareto curve of cache miss ratio at different device-level write rates under 16 GB DRAM and 2 TB flash capacity. At very low write rates, LS is best, but it is limited by DRAM from scaling to larger caches. Thus, for most write rates, Kangaroo outperforms both systems because it can take advantage of the entire cache capacity, has a lower write rate than SA, and has a better eviction policy than the other two systems.

Figure 9

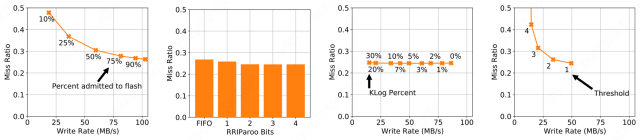


(a) Facebook *(b)* Twitter

Fig. 11. Pareto curve of cache miss ratio vs. average object size. Object sizes are limited to $[1\,B, 2048\,B]$. The write rate is constrained to 62.5 MB/s for a 2 TB flash drive with 16 GB of DRAM.

Figure 10

5.4 Parameter sensitivity and benefit attribution



(a) Probabilistic admission. *(b)* + RRIParoo *(c)* + KLog. *(d)* + Threshold.

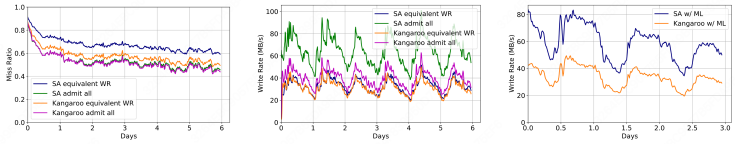
Fig. 12. Miss ratio vs. application-level write rate based on various design parameters in Kangaroo: (a) KLog admission probability, (b) RRIParoo metadata size, (c) KLog size (% of flash-device capacity), and (d) KSet admission threshold.

Figure 11

5.5 Production deployment test

we present results from two production test deployments on a small-object workload at Facebook, comparing Kangaroo to SA. Each deployment receives the same request stream as production servers but does not respond to users. Due to limitations in the production setup, we can only present application-level write rate (i.e., not device-level) and flash miss ratio (i.e., for requests that miss in the DRAM cache). In addition, both systems use the same cache size (i.e., Kangaroo does not benefit from reduced over-provisioning).

To find appropriate production configurations, we chose seven configurations for each system that performed well in simulation: four with probabilistic pre-flash admission and three with a machine-learning (ML) pre-flash admission policy. The first production deployment ran all probabilistic admission configurations and the second ran all ML admission configurations. Since these configurations ran under different request streams, their results are not directly comparable.



(a) Flash miss ratio. *(b)* Application flash write rate. *(c)* ML admission.

Fig. 13. Results from two production test deployments of Kangaroo and SA, showing (a) flash miss ratio and (b) application flash write rate over time using pre-flash random admission and (c) application flash write rate over time using ML admission. With random admission at equivalent write-rate, Kangaroo reduces misses by 18% over SA. When Kangaroo and SA admit all objects, Kangaroo reduces write rate by 38%. With ML admission, Kangaroo reduces the write rate by 42.5%.

Figure 12

6 Problems

7 References